TERM PROJECT REPORT

A Comparative Study of Open-Source Text Analytics Libraries

NLTK, spaCy and Stanford CoreNLP

Features, Ease of Use and Performance

Subject: Text Analytics

Submitted by

Santhana Krishnan S

Register No.: _____

Master of Computer Applications (2025 - 2027)

Ayya Nadar Janaki Ammal College (Autonomous), Sivakasi

Under the guidance of: _____

Date: 2026-10-19

Abstract

Text analytics converts unstructured text into structured information, and most projects start by choosing a software library. This report compares three widely used open-source toolkits, NLTK, spaCy and Stanford CoreNLP, on three dimensions: the features they provide, how easy they are to use, and how they perform. The same tasks (tokenization, part-of-speech tagging, lemmatization and named-entity recognition) were run on the same 100 movie reviews (380,806 characters), and POS accuracy was measured on 500 Penn Treebank sentences. On our test machine NLTK was fastest for tokenization and POS-only processing, but when lemmatization and NER were added spaCy was about 11 times faster than NLTK (12 times with batching). POS accuracy was 96.58% for NLTK and 95.88% for spaCy. CoreNLP offers the deepest analysis (constituency parsing, coreference, OpenIE) at the cost of a Java server and a GPL licence. We conclude that NLTK suits learning, spaCy suits building applications, and CoreNLP suits tasks needing deep linguistic analysis.

Keywords: text analytics, NLP, NLTK, spaCy, Stanford CoreNLP, benchmark.

1. Introduction

Natural Language Processing (NLP) libraries provide ready-made components such as tokenizers, taggers and parsers. Choosing among them affects development time, accuracy, speed and deployment cost. NLTK (Natural Language Toolkit) was created for teaching and research and bundles many corpora and algorithms. spaCy is an industrial-strength Python/Cython library designed for fast, production use. Stanford CoreNLP, written in Java, provides a broad suite of linguistic annotators developed by the Stanford NLP Group.

1.1 Objectives

(i) Compare the features of NLTK, spaCy and Stanford CoreNLP. (ii) Compare their ease of use in installation, code size and documentation. (iii) Compare their performance in speed, POS-tagging accuracy, start-up time and memory. (iv) Recommend the right library for typical use cases.

1.2 Scope

The study covers English text and four core tasks: tokenization with sentence splitting, POS tagging, lemmatization and named-entity recognition. Machine-learning model training and non-English languages are outside the scope.

2. Overview of the Libraries

	NLTK	spaCy	Stanford CoreNLP
Origin	University of Pennsylvania (2001)	Explosion AI (2015)	Stanford NLP Group
Language	Python	Python / Cython	Java

	NLTK	spaCy	Stanford CoreNLP
Licence	Apache 2.0	MIT	GPL v3 (commercial licence available)
Philosophy	Teaching and research toolkit	Fast, production-ready pipelines	Deep linguistic analysis
Version used	3.10.3	3.8.16 (model en_core_web_sm)	4.5.x (see notebook)

3. Methodology

Environment. Python 3.12.3, 1 CPU core, Linux-6.18.44-fc-v139-x86_64-with-glibc2.39. All code is in the accompanying Jupyter Notebook.

Data. Speed was measured on 100 movie reviews from the NLTK *movie_reviews* corpus (average 3,808 characters). Accuracy was measured on the first 500 sentences (11,784 tokens) of the NLTK Penn Treebank sample, with traces removed. All three libraries use the Penn Treebank tag set, so tags are directly comparable; pre-tokenized input was supplied so token boundaries match the gold data.

Pipelines timed. (a) tokenization with sentence splitting; (b) tokenization + POS tagging; (c) tokenization + POS + lemmatization + NER. Each document was processed in its own call; one warm-up call was made; the median of repeated runs is reported (3 runs for (a) and (b), 2 for (c) with NLTK). For spaCy, the dependency parser was disabled in (c) to match the other libraries; a batched run using *nlp.pipe* is also reported.

Start-up and memory. Each library was loaded in a fresh process and the increase in resident memory and the time to load and run one sentence were recorded.

4. Feature Comparison

Table 1 summarizes what each library provides out of the box. "Partial" means the capability exists only in limited form or needs an add-on.

Feature	NLTK	spaCy	CoreNLP
Implementation language	Python	Python / Cython	Java (REST, Python wrappers)
Licence	Apache 2.0	MIT	GPL v3 (commercial licence available)
Tokenization + sentence split	Yes	Yes	Yes
Stemming	Yes (Porter, Snowball, Lancaster)	No (lemmas only)	No (lemmas only)
Lemmatization	Yes (WordNet, POS needed)	Yes	Yes
POS tagging	Yes	Yes	Yes
Named-entity recognition	Partial (basic chunker)	Yes (18 types)	Yes (incl. dates, numbers via SUTime)
Dependency parsing	Partial (no pretrained model)	Yes	Yes

Feature	NLTK	spaCy	CoreNLP
Constituency parsing	Partial (no pretrained model)	No (plugin)	Yes
Coreference resolution	No	Partial (experimental)	Yes
Sentiment analysis	Yes (VADER lexicon)	No (add-on)	Yes (neural)
Open information extraction	No	No	Yes (OpenIE)
Pretrained word vectors	No	Yes (md / lg models)	No
Built-in corpora / lexicons	Yes (WordNet, Brown, Treebank ...)	No	No
Train custom models	Yes	Yes (config system)	Yes (e.g. CRF NER)
Languages with pretrained models	Limited tokenizers / stopwords	~25 pipelines, 70+ tokenizers	8 (EN, AR, ZH, FR, DE, HU, IT, ES)
Primary design goal	Teaching / research	Production speed	Linguistic depth

Table 1: Feature comparison (compiled from official documentation).

NLTK is unique in shipping corpora and lexicons (WordNet, Brown, Treebank, VADER) and stemmers. spaCy supplies a coherent, pretrained pipeline including a dependency parser and 18 entity types, plus word vectors in larger models. CoreNLP is the only one of the three with built-in constituency parsing, coreference resolution and OpenIE.

5. Ease of Use

Aspect	NLTK	spaCy	CoreNLP
Install	pip install nltk	pip install spacy	Download ~0.5 GB zip + JDK
Extra downloads	Per-task data via nltk.download (6+ packages here)	1 model (python -m spacy download)	Models bundled in the zip
Lines for the common job	8	6	5
Needs a running server	No	No	Yes (HTTP, JVM)
Output object	Lists / tuples / Trees	Doc, Token, Span objects	JSON
Documentation	Free book + API docs	Extensive guides + course	Reference docs, fewer tutorials

Table 2: Ease-of-use comparison.

For the same job (tokens, POS tags, lemmas, entities for one text) NLTK needed 8 lines, spaCy 6 lines and CoreNLP 5 lines. NLTK needs several separate data downloads and a manual POS-tag conversion before WordNet lemmatization. spaCy returns rich Doc/Token objects, so everything comes from one call. CoreNLP requires Java, a large download and a running server, and returns JSON that must be unpacked, although this also makes it usable from any language.

Author's rating (1 = poor, 5 = excellent)

Criterion	NLTK	spaCy	CoreNLP
Feature breadth	3	4	5
Ease of use	4	5	3
Learning / teaching	5	4	3
Production readiness	2	5	4

Table 3: Qualitative scores (subjective, based on Sections 4 and 5).

6. Performance Results

6.1 Speed

Library	Task	Docs/sec	Tokens/sec	Total sec
NLTK	Tokenization	420.84	316,376	0.238
NLTK	+ POS tagging	45.2	33,982	2.212
NLTK	+ Lemma + NER	1.38	1,035	72.65
spaCy	Tokenization	337.39	269,305	0.296
spaCy	+ POS tagging	27.95	22,309	3.578
spaCy	+ Lemma + NER	15.18	12,114	6.589
spaCy (nlp.pipe)	+ Lemma + NER	16.97	13,544	5.893
Stanford CoreNLP	all	not measured	not measured	-

Table 4: Processing speed on 100 reviews.

Note on CoreNLP. The environment used to generate this version of the report had no access to the Stanford download site, so CoreNLP could not be run and no CoreNLP numbers are reported. Running the notebook on Google Colab or a machine with Java adds the CoreNLP rows automatically; re-run *build_report.py* to include them.

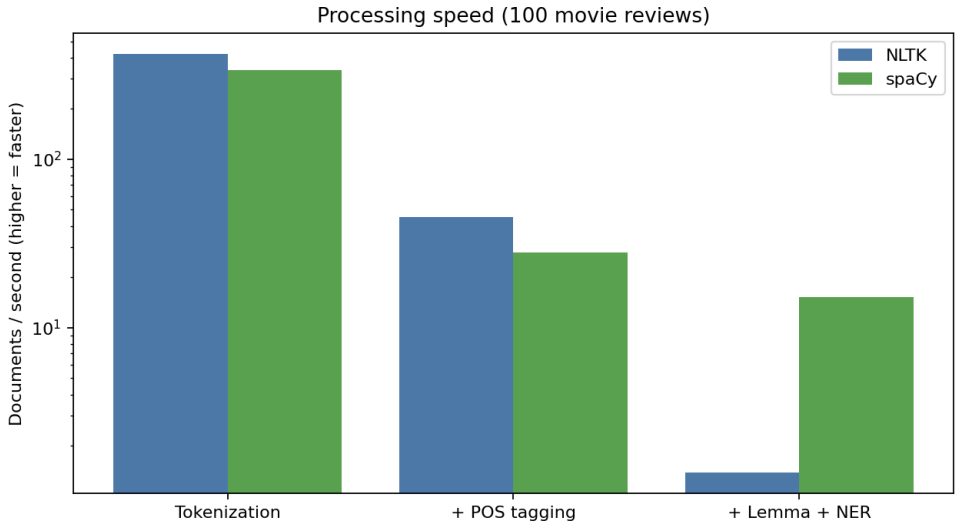


Figure 1: Documents processed per second (log scale).

Tokenization and POS-only processing were fast in both libraries, with NLTK ahead on this machine (45.2 vs 27.95 documents/s for POS). The picture reverses once NER is included: NLTK's *ne_chunk* is expensive, so NLTK fell to 1.38 documents/s while spaCy reached 15.18 (16.97 with *nlp.pipe*), a speed-up of about 11 times (12 times batched). spaCy also produces richer, neural-model-based entity labels in that time.

6.2 POS-tagging accuracy

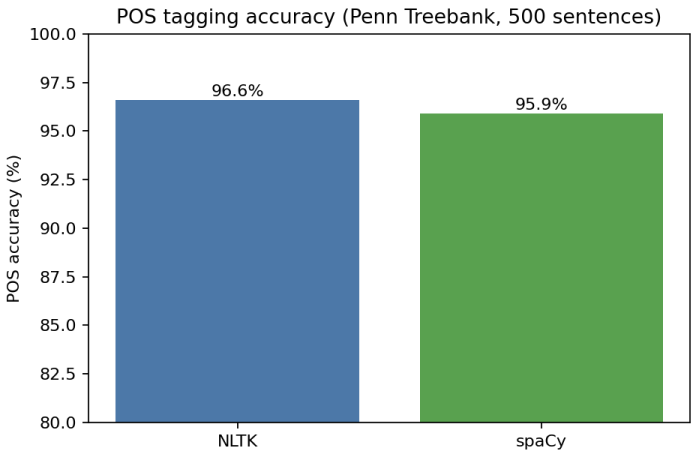


Figure 2: POS accuracy on 500 Penn Treebank sentences.

NLTK scored 96.58% and spaCy 95.88%. The gap is small, and the Treebank sample overlaps with the training data of some taggers, so the figures are optimistic and should be read only as a relative comparison. The small spaCy model trades a little accuracy for speed and size.

6.3 Start-up time and memory

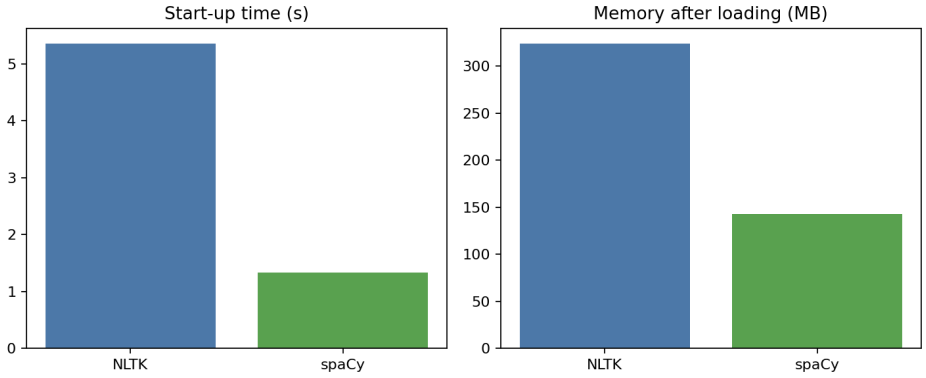


Figure 3: Time to load and extra memory used.

NLTK took 5.36 s and 324 MB to load the tagger, chunker and WordNet, while spaCy took 1.33 s and 143 MB. CoreNLP runs in a separate JVM and typically needs a few gigabytes of heap for the full annotator set, but this was not measured here.

7. Discussion

Features. CoreNLP is broadest in linguistic depth, spaCy gives the most complete ready-to-use pipeline in Python, and NLTK gives the most educational material and classical tools such as stemmers and lexicons. **Ease of use.** spaCy needed the fewest steps and lines; NLTK is simple per function but needs more glue code and data downloads; CoreNLP has the highest set-up cost. **Performance.** Results depend on the task. Speed in simple tasks does not predict speed in a full pipeline, so benchmarks must use the pipeline that the application actually needs. **Licensing and deployment.** NLTK (Apache 2.0) and spaCy (MIT) are easy to use commercially; CoreNLP is GPL v3 and needs a commercial licence for closed-source products.

Recommendations

Use case	Suggested library
Learning NLP concepts, classroom work, corpora and lexicons	NLTK
Production pipelines, information extraction, web services	spaCy
Coreference, constituency parsing, OpenIE, temporal expressions	Stanford CoreNLP
Quick stemming or VADER sentiment baseline	NLTK

8. Conclusion

Each library has a clear role. NLTK is the best learning toolkit, spaCy offers the best balance of speed, simplicity and accuracy for applications, and CoreNLP provides the deepest analysis when those extra annotators are required. The measurements showed that the faster library depends on the task: NLTK was ahead for tokenization and POS only, while spaCy was far faster once NER and lemmatization were included.

Limitations and future work

The study used 100 documents, one machine, English text and the small spaCy model. The Treebank sample overlaps with training data of some taggers. CoreNLP timings include HTTP and JSON overhead. Future work: add the transformer-based spaCy model and Stanza, evaluate NER against a labelled dataset such as CoNLL-2003, and test larger corpora and multiple languages.

References

[1] Bird, S., Klein, E., and Loper, E. (2009). Natural Language Processing with Python. O'Reilly Media.

[2] Honnibal, M., Montani, I., Van Landeghem, S., and Boyd, A. (2020). spaCy: Industrial-strength Natural Language Processing in Python. Zenodo.

[3] Manning, C. D., Surdeanu, M., Bauer, J., Finkel, J., Bethard, S. J., and McClosky, D. (2014). The Stanford CoreNLP Natural Language Processing Toolkit. Proceedings of ACL: System Demonstrations, pp. 55-60.

[4] Marcus, M. P., Santorini, B., and Marcinkiewicz, M. A. (1993). Building a large annotated corpus of English: the Penn Treebank. Computational Linguistics, 19(2).

[5] NLTK documentation: https://www.nltk.org

[6] spaCy documentation: https://spacy.io

[7] Stanford CoreNLP documentation: https://stanfordnlp.github.io/CoreNLP/